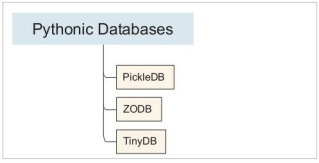

Figure 1: Python databases

The following code segment illustrates the basics of using pickleDB.

```
>>> import pickledb

>>> db = pickledb.load('example.db', False)

>>> db.set('key', 'value')
True

>>> db.get('key')
'value'

>>> db.dump()
True
```

Some of the popularly used commands of pickleDB are explained below.

- *LOAD path dump:* This is used to load a database from a file.
- *SET key value:* This is the value of a key with the string.
- *GET key:* Used to retrieve the value of the key.
- *GETALL:* Used to fetch all the keys in a database.
- *REM key:* Deletes the key.
- *DUMP:* Saves the database from the memory into the file specified with the load command.

Apart from the above mentioned commands there are various other commands. Interested readers can get the complete details from *https://pythonhosted.org/pickleDB/commands.html*.

pickleDB can be used for those scenarios in which the key-value store type of format is suitable.

TinyDB

As the name indicates, TinyDB is a compact, lightweight database and is document-oriented. It is written 100 per cent in Python and has no external dependencies. As the official documentation says, TinyDB is a database optimised for your happiness.

The applications which are best suited to TinyDB are small apps for which a traditional SQL-DB server based approach would be an overload. The major features of

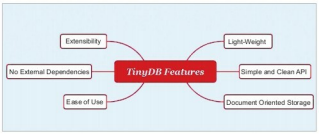

Figure 2: Tiny DB features

TinyDB are listed below:

- As the name indicates, TinyDB is very small. The complete source code is only 1200 lines.
- TinyDB is based on document-oriented storage. This type of storage is adopted in popular tools such as MongoDB.
- The API of TinyDB is very simple and clean. Primarily, TinyDB has been designed keeping ease of usage in mind.
- Another likeable feature of TinyDB is the non-dependency. In addition, TinyDB supports all the recent versions of Python. It works on Python 2.6, 2.7, 3.3 – 3.5.
- Extensibility is another major feature of TinyDB. With the help of middleware, TinyDB behaviour can be extended to suit specific needs.

Though TinyDB has various advantages, it is not a single-size-fits-all solution for problems. It has certain limitations as listed below:

- TinyDB is not suitable for those scenarios in which high speed data retrieval is the key.
- If you need to access the database from multiple processes or threads, then TinyDB won't be optimal. Similarly, HTTP server based access is another scenario in which it won't be suitable.

Having seen the pros and cons of TinyDB, let us look at how to use it. As is the case with many other packages, TinyDB can be installed simply with the following command:

```
$ pip install tinydb
```

The following code snippet explores how to create and store the values in TinyDB:

```
from tinydb import TinyDB, Query
db = TinyDB('db.json')
db.insert({'type': 'OSFY', 'count': 700})
db.insert({'type': 'EFY', 'count': 800})
```

After the successful execution of the above mentioned snippet, you can retrieve the values as shown below.

To list all values, give the following commands:

```
db.all()
[{'count': 700, 'type': 'OSFY'}, {'count': 800, 'type':
'EFY'}]
```